BACKEND / JAVA 25

Symplifica News

Arquitectura lógica, flujo de procesamiento y componentes del sistema

2 NOV 2026

PROPÓSITO DEL SISTEMA

Consultar de forma automatizada el canal RSS de noticias de Portafolio, detectar artículos nuevos mediante la comparación de sus identificadores únicos (GUID), generar un resumen consolidado en español mediante inteligencia artificial local (Ollama + Gemma 3 4B), despacharlo por correo HTML y auditar el resultado completo en PostgreSQL.

1. ARQUITECTURA LÓGICA DE SYMPLIFICA NEWS



2. ESTRUCTURA DE PAQUETES Y RESPONSABILIDADES

Paquete / Capa	Responsabilidad en el Sistema
`config / job`	Configuración de integración con Ollama (OllamaConfig) y tareas programadas en segundo plano con JobRunr (NewsJob).
`controller / dto`	Endpoints REST para ejecución e historial. Transferencia de datos mediante récords como `NewsResponse`.
`entity / repository`	Entidades JPA (`NewsLog` , `NewsItem`) y repositorios Spring Data para persistencia.
`service / rss`	Descarga, parseo de XML a objetos `RssItem` (título, enlace, descripción, fecha, etc.).
`service / ai & email`	Generación del resumen con LangChain4j + Ollama y armado de correo HTML distribuido por SMTP.
`service / logging`	Registro de estado de ejecuciones (`RUNNING` , `SUCCESS` , `FAILED`) y métricas de procesamiento.
`exception`	Manejo centralizado de errores de la API devolviendo estructuras bajo el estándar `ProblemDetail`.

MODELO DE DATOS & API

Persistencia, DTOs y Operaciones

Estructura del modelo Java, migraciones Flyway y endpoints REST

2 NOV 2026

3. REGISTRO DTO: NEWSRESPONSE

```
package com.symplifica.symplifica_news.dto;
import java.time.OffsetDateTime; public record
NewsResponse( Long id, String guid, String title,
String link, String description, String
publicationDate, String category, String imageUrl,
OffsetDateTime processedAt, Long newsLogId ) {}
```

4. MIGRACIONES FLYWAY

Versión	Descripción de la Migración
V1	Tabla `users` para posibles ampliaciones.
V2	Tabla `news_log` (registro global de ejecución).
V3	Adición del estado `RUNNING` a las ejecuciones.
V4	Tabla `news_items`, índices y FK a `news_log`.

5. BASE DE DATOS RELACIONAL (POSTGRESQL)

Tabla	Propósito Principal	Campos Clave & Reglas
`news_log`	Auditoría completa de cada ciclo del trabajo automático o manual.	Fechas de inicio/fin, estado (`RUNNING` , `SUCCESS` , `FAILED`), total de noticias, resumen generado y mensaje de error.
`news_items`	Representación persistida de cada noticia procesada.	`guid` (índice único para deduplicación), `title` , `link` , `description` , `publication_date` , `category` , `image_url` , `processed_at` y FK `news_log_id` .

6. API REST DEL SERVICIO

Método	Endpoint	Respuesta / DTO	Descripción
`GET`	`/api/news/logs`	`List<NewsLogResponse>`	Retorna las últimas 50 ejecuciones registradas.
`GET`	`/api/news/logs/{id}`	`NewsLogDetailResponse`	Detalle individual de una ejecución y sus noticias asociadas.
`GET`	`/api/news/items`	`List<NewsResponse>`	Últimas 100 noticias procesadas estructuradas con el DTO.
`POST`	`/api/news/run`	`NewsLogResponse`	Ejecución síncrona manual del flujo completo y envío de correo.

7. DECISIONES OPERATIVAS, TECNOLOGÍAS Y LÍMITES

Stack Tecnológico

- **Java 25** & **Spring Boot 4.1.2**
- **PostgreSQL** + Migraciones Flyway
- **JobRunr 9.0.1** (Dashboard en port 8000)
- **LangChain4j 1.23.0** + Ollama (Gemma 3 4B)

Consideraciones Técnicas

- **Deduplicación:** Se realiza por GUID antes del procesamiento con IA. Si no hay ítems nuevos, la ejecución se marca SUCCESS con 0 noticias.
- **Transaccionalidad:** SMTP y PostgreSQL operan en pasos secuenciales (no transacciones distribuidas).